



AI LABS / DESIGN SYSTEM

Step-by-step user & release guide

[Figma](#) / [Installation](#) / [GitHub](#) / [npm](#) / [Recovery](#)

Angular components, tokens, and the component showcase

Repository baseline 0.1.2 | Commit 88d2cc8

22 September 2026

DESIGN > CODE > REVIEW > RELEASE > ADOPT

A source-checked reference for designers, developers, and maintainers.

Includes practical limits, verification steps, and official workflow references.

Contents

Click a section title to jump to it. PDF bookmarks provide the same navigation.

1. Start here: what gets updated where	3
2. Use the showcase application	4
3. Add the Figma library to your design file	5
4. Install and use the Angular library	6
5. Run and package the repository locally	8
6. Add to or change the component library	9
7. Publish and adopt Figma updates	10
8. Update GitHub and deploy the showcase	10
9. Release an npm package update	11
10. Upgrade a consuming application	13
11. Update npm and project dependencies	13
12. Validation checklist and release handoff	14
13. Troubleshooting and recovery	15
14. Evidence and maintenance of this guide	16

Baseline: package 0.1.2 | Angular 22 | 22 September 2026 **Audience:** designers, application developers, library maintainers, release owners **Source baseline:** repository commit 88d2cc8

1. Start here: what gets updated where

This project contains a reusable Angular component library and a browser showcase. The Figma design library, GitHub source, GitHub Pages website, npm package, and each consuming application have separate update steps. The website does not provide an administration screen for publishing any of them.

Your goal	Follow this procedure	Result
Explore the application	Section 2	Browse tokens and copy component examples
Add the design library to a Figma file	Section 3	Use linked component instances
Install the Angular library	Section 4	Styled components in your application
Run this repository locally	Section 5	Editable component showcase
Add or modify library components	Section 6	Reviewed source and design changes
Publish Figma updates	Section 7	Updates available to design consumers
Update GitHub and the live demo	Section 8	Source merged and Pages deployed
Publish an npm version	Section 9	New installable package version
Upgrade a consuming application	Section 10	Application adopts the release
Update project dependencies	Section 11	Reviewed dependency and lockfile changes
Troubleshoot or recover	Sections 12-13	Diagnosis and rollback instructions

Project links and access

- [Live showcase](#)
- [GitHub repository](#)
- [Figma source file linked by the application](#)
- [npm package page](#)

The Figma link is recorded in the source code. Its current contents, publication state, and your permissions were not inspected for this documentation. Registry ownership and the currently published npm version were not verified. Version 0.1.2 is the local package baseline; verify registry versions before installation or release.

Task	Access needed
Browse the website	Browser
Add a Figma library	Edit access to destination file, view access to source library, available published library

Task	Access needed
Publish Figma changes	Edit access to source file and eligible Figma plan/seat
Work on source	Git, compatible Node/npm, local repository
Push source and merge	Repository write access or fork/PR workflow
Configure Pages	Repository settings permission
Publish npm	npm account with package publishing rights and required authentication

Command blocks state their working directory. Replace angle-bracket placeholders before running a command; they are not literal shell arguments. Release version 0.1.3 below is an example next patch release, not a promise that it is available.

2. Use the showcase application

1. Open the live showcase or run it locally using Section 5.
2. Use **Overview** for the introduction and install command. **Browse components** scrolls to the catalog.
3. Open **Foundations** for design principles, then **Colors** to inspect ramps. Click a color swatch to copy its displayed value.
4. Open **Variables** for semantic colors and spacing, radius, and stroke reference. Prefer semantic names when implementing an interface.
5. Open **Components** to compare variants and states. **Open Figma library** opens the linked design file.
6. Open **Playground**, choose a component, and change the available variant, size, state, and toggle controls. The preview and Angular template update together.
7. Select **Copy code**. Import the indicated component from `ailabs-design-system` and add it to the consuming standalone component's `imports` array.
8. Replace demo content, wire application state and event handlers, and test your actual interaction.
9. Switch the header theme control between light and dark. Reload to check that your browser keeps the preference.

Expected result: interactive examples render, copied code matches the selected controls, and theme preference survives a reload when local storage is available.

The overview copies `npm install ailabs-design-system`; this command alone does not register the required CSS. Follow Section 4. The header GitHub icon is a working link; the hero **View on GitHub** button has no navigation handler in the current source.

The playground offers Button, Chip, Badge, Checkbox, Switch, Radio, Text field, Tabs, Menu item, List item, Search, and Toolbar. Other exported components are described in the technical reference. Copied SVG `<use>` references such as `#icon-sparkles` and `#icon-material-format-bold` refer to symbols defined in the showcase. Supply those symbols or replace them with your own icons in a consuming app.

3. Add the Figma library to your design file

3.1 Enable an already published library

1. Open your destination Figma Design file with edit permission.
2. Select **Assets** in the left sidebar, then the **Libraries** icon.
3. Search for the source file's library name, for example **AILab Design System**. Confirm the owner and source file before selecting it.
4. Open the library details and select **Add to file**.
5. Close the dialog. Drag a component from **Assets** onto the canvas.
6. Select the instance and choose its available properties or variants in the right sidebar.
7. Apply published styles and variables through the relevant property picker.

Expected result: an instance of the source component appears in the destination file and remains connected to its main component. Keep instances linked to receive future updates.

A library must be published before another file can add it. Figma currently documents library access on paid plans; users need view access to the library and edit access to the destination. See [Figma: add or remove a library](#).

3.2 If the library is missing

1. Open the linked source file and confirm access. Request access from its owner if needed.
2. Confirm that the source is published to a team or workspace you can access.
3. Ask a maintainer to follow Section 7 if it has never been published.
4. Check that the required assets were included in the published selection.
5. If working from an authorized duplicate, record that it is a separate library. Changes to the original will not automatically synchronize into your duplicate.

3.3 Match design variants to code

Design choice	Angular implementation
Primary / secondary / outline / ghost button	ButtonComponent with variant
Small / medium / large button	size="sm", "md", or "lg"
Filled / outlined chip	ChipComponent with variant
Selected chip	[selected] plus consumer selection handler
Text field	InputComponent, selector ds-input
Standard / vibrant toolbar	ToolbarComponent with tone
Docked / floating toolbar	variant="docked", "floating-horizontal", or "floating-vertical"
Semantic surface or text variable	Matching CSS semantic custom property

This mapping is based on the checked-in Angular API. Validate actual Figma property names in the source file. The repository contains no automatic Figma synchronization or Code Connect mapping files.

4. Install and use the Angular library

4.1 Check compatibility

1. Open a terminal at your consuming Angular workspace root, beside `angular.json`.
2. Run the commands below. The package declares Angular `>=22.0.0 <23.0.0` peers.

```
node --version
npm --version
npx ng version
npm view ailabs-design-system versions --json
npm view ailabs-design-system@0.1.2 peerDependencies
```

Use an available version whose peer dependencies match your app. If 0.1.2 is unavailable, use the local archive procedure in Section 5 or select an appropriate published version. Do not bypass peer dependency errors as an installation strategy.

4.2 Recommended installation with Angular CLI

In the consuming workspace, after confirming that version 0.1.2 exists:

```
npx ng add ailabs-design-system@0.1.2
```

To target only one application in a multi-project workspace:

```
npx ng add ailabs-design-system@0.1.2 --project my-app
```

Replace `my-app` with the application key from `angular.json`. Without `--project`, the schematic configures all application projects with build options. It prepends `ailabs-design-system/styles.css` to the base build styles array and avoids duplicate entries. Restart the development server afterward.

4.3 Manual npm installation

1. In the consuming workspace, install the selected version.

```
npm install ailabs-design-system@0.1.2
```

2. Edit the correct application's `architect.build.options.styles` in `angular.json`, or its equivalent `targets.build.options.styles` location. Keep the existing entries and add the library before application overrides.

```
"styles": [
  "ailabs-design-system/styles.css",
  "src/styles.css"
]
```

3. Inspect configuration-specific style arrays too; if a configuration overrides `styles`, include the library there. The schematic updates base build options only.
4. Restart the development server. Confirm that the package appears in `package.json` and the lockfile.

4.4 Build your first screen

Create or adapt a standalone component in the consuming app:

```
import { Component } from '@angular/core'
import {
  ButtonComponent, InputComponent, CheckboxComponent,
} from 'ailabs-design-system'

@Component({
  selector: 'app-example',
  standalone: true,
  imports: [ButtonComponent, InputComponent, CheckboxComponent],
  template: `
    <ds-input label="Project name" [(value)]="projectName" />
    <ds-checkbox [(checked)]="includeInsights">
      Include insights
    </ds-checkbox>
    <button dsButton type="button" (click)="save()">Save</button>
    <p>{{ status }}</p>
  `,
})
export class ExampleComponent {
  projectName = ''
  includeInsights = false
  status = ''

  save(): void {
    this.status = `Saved ${this.projectName}`
  }
}
```

Render this component from your app root or route. Import `ExampleComponent` wherever its selector is used. Run your app's build and serve scripts, type a name, toggle the checkbox, and select Save.

Expected result: controls are styled and the displayed message updates. This example changes local state only; saving to a backend belongs to your application.

4.5 Forms, themes, and toolbars

The custom controls do not implement `ControlValueAccessor`. Bind `value/valueChange` or `checked/checkedChange`; do not expect `formControlName` or `ngModel` to work directly. For reactive forms, connect the events explicitly or implement an adapter. Boolean inputs should use property bindings such as `[disabled]="true"`.

To use the packaged dark theme in a browser-only application, set `data-theme="dark"` on the root `<html>` element. Use `data-theme="light"` or remove the attribute for light defaults. The consumer must implement any toggle or persistence; those behaviors live in the showcase, not the library.

Import both `ToolbarComponent` and `ButtonComponent` for a toolbar with button children:

```
<ds-toolbar variant="floating-horizontal" tone="vibrant"
  ariaLabel="Editor formatting">
  <button dsButton type="button" toolbarLeading>Bold</button>
  <button dsButton type="button" variant="ghost">Italic</button>
  <button dsButton type="button" toolbarTrailing
    variant="ghost">Underline</button>
</ds-toolbar>
```

Add click handlers for real actions. `[expanded]="false"` hides leading and trailing content on floating toolbars; docked toolbars keep both groups. Floating variants provide appearance and layout, not drag behavior or automatic viewport placement.

5. Run and package the repository locally

5.1 Clone and start

The installed CLI declares Node ^22.22.3, ^24.15.0, or >=26.0.0. This documentation build used Node 24.16.0 and npm 11.13.0. Check the [Angular compatibility reference](#) when changing framework versions.

In your chosen parent folder:

```
git clone https://github.com/ItsVivekRamachandran/Ailabs-DesignSystem.git
cd Ailabs-DesignSystem
npm ci
npm start
```

Open <http://localhost:4200/>. If you already have this checkout, use its existing directory instead of cloning again. Stop the server with Ctrl+C. If port 4200 is busy, run `npm start -- --port 4300` and open port 4300.

Expected result: the six-section showcase appears and source changes trigger rebuilds.

5.2 Understand the build commands

Command in repository root	Output or behavior
<code>npm run build</code>	Angular library in <code>dist/ailabs-design-system</code>
<code>npm run pack</code>	Builds library, then creates versioned <code>.tgz</code> in <code>dist</code>
<code>npm run build:demo</code>	Showcase in <code>dist/playground/browser</code>
<code>npm run build:pages</code>	Showcase with <code>/Ailabs-DesignSystem/</code> base href
<code>npm run watch</code>	Development library rebuilds
<code>npm run watch:demo</code>	Development showcase rebuilds

5.3 Test the local package in another app

1. In the library repository, run `npm run pack`.
2. For baseline 0.1.2, find `dist/ailabs-design-system-0.1.2.tgz`. Use the actual version if it has changed.
3. In a separate Angular 22 consuming app, install the archive using an absolute path.

```
npm install /absolute/path/to/itsproject/dist/ailabs-design-system-0.1.2.tgz
```

4. Add the stylesheet manually as in Section 4, import a button and an input, and build the app.
5. Check both themes and at least one output event.

For quick iteration, `npm install /absolute/path/to/itsproject/dist/ailabs-design-system` is also possible. Use the archive for release validation because it reflects packaged contents. Do not edit generated `dist` files; rebuild them from source.

6. Add to or change the component library

6.1 Begin a change

1. Agree on the intended component behavior, variants, inputs, outputs, states, and accessibility requirements.
2. Record the Figma source link and affected component or variable names in the work item.
3. Start from a clean repository worktree. Commit or otherwise preserve unrelated work first.

```
git switch main
git pull --ff-only origin main
git switch -c feature/ds-empty-state
```

4. Make the design change in the main Figma component or an agreed design working copy. Coordinate its publication with the code release.

6.2 Add an Angular component: worked example

Create `src/app/design-system/empty-state.component.ts`:

```
import { ChangeDetectionStrategy, Component, Input } from '@angular/core'

@Component({
  selector: 'ds-empty-state',
  standalone: true,
  host: { class: 'ds-empty-state' },
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <h3>{{ title }}</h3>
    <p>{{ description }}</p>
    <ng-content />
  `,
})
export class EmptyStateComponent {
  @Input() title = 'No results'
  @Input() description = 'Try another search.'
}
```

1. Add styles to `src/design-system/components.css` using existing semantic tokens.

```
.ds-empty-state {
  padding: var(--spacing-24);
  color: var(--text-primary);
  background: var(--surface-default);
}
```

2. Export the class from `src/app/design-system/index.ts`.

```
export { EmptyStateComponent } from './empty-state.component'
```

3. `src/public-api.ts` already re-exports this barrel, so no second per-component export is needed there.
4. Import `EmptyStateComponent` into `src/app/app.component.ts` and its component imports list.
5. Add a catalog example in `src/app/app.component.html`.
6. For playground support, update `playgroundComponents`, the `playgroundCode` computed switch, selection defaults if needed, and the template controls and preview switch.
7. Document defaults, supported states, projection slots, events, and limitations in the technical reference.
8. Build the library and showcase. Install the archive into a consumer to verify that the new export and CSS are packaged.

The example above is a recipe, not a component already included in version 0.1.2.

6.3 Change a design token

1. Identify the semantic meaning and affected Figma variable. Record the current and proposed value or alias.
2. Update `src/design-system/tokens.css`; add corresponding dark-theme overrides when theme-sensitive.
3. Update `src/design-system/tokens.ts` for metadata rendered in the showcase. These files are manually maintained.
4. Update the matching Figma variable or alias, checking the actual collection and mode names in that file.
5. Inspect every affected component in light and dark themes. Check contrast, focus treatment, and disabled states.
6. Record the mapping and release note. Avoid renaming/removing a public token without a consumer migration plan.

7. Publish and adopt Figma updates

7.1 Publish the source library

1. Open the linked source file with edit permission. Ensure it contains the intended components, styles, or variables.
2. If needed, move the file from Drafts to an appropriate folder and confirm the eligible plan and Full seat required by Figma.
3. Open **Assets > Libraries**. Under **This file**, select **Publish**.
4. Review the changed assets and exclude unfinished work.
5. Describe the change and include the matching code release or PR reference.
6. Select the appropriate publishing destination when offered, then **Publish**.
7. Confirm the success notification and record the Figma version/history reference.

This publication is separate from GitHub and npm. See [Figma: publish a library](#) for current eligibility and controls.

7.2 Accept updates in a consuming design

1. Open the destination design file containing linked instances.
2. Open **Review library updates**, then **Updates**.
3. Enable **Show updates for all pages** when reviewing the full file.
4. Inspect the available changes and compare affected instances.
5. Choose **Update** for individual assets or **Update all** when all changes are appropriate.
6. Review important screens after the update, including overrides, text wrapping, and resized layouts.

Expected result: the selected linked assets use the published revision. Detached instances require separate maintenance. See [Figma: review and accept updates](#).

8. Update GitHub and deploy the showcase

8.1 Validate and create a pull request

In the repository root:

```
npm run build
npm run build:pages
git diff --check
git status --short
git diff
```

1. Review the changes and run the manual checks in Section 12.

2. Stage only intended files using `git add <changed-file-paths>`; replace the placeholder with explicit paths.
3. Commit and push the working branch.

```
git commit -m "feat: add empty state component"
git push -u origin feature/ds-empty-state
```

4. On GitHub, open a pull request to main. Explain the problem, behavior change, API impact, Figma reference, and validation results.
5. Resolve review feedback and merge according to repository rules. A merge to main triggers the Pages workflow.

8.2 Configure Pages once

1. Open the repository's **Settings > Pages**.
2. Under **Build and deployment**, set **Source** to **GitHub Actions**.
3. Confirm Actions is enabled and any github-pages environment requirements can be satisfied.

This matches the checked-in custom workflow. See [GitHub: configure a Pages publishing source](#).

8.3 Observe deployment

1. Open **Actions > Deploy Angular app to GitHub Pages**.
2. Inspect the run for your merged commit. Alternatively, choose **Run workflow** on main for a manual run.
3. The build job checks out source, selects Node 22, runs `npm ci` and `npm run build:pages`, copies `index.html` to `404.html`, and uploads `dist/playground/browser`.
4. The deploy job publishes that artifact to the `github-pages` environment.
5. Open the deployment URL and verify the intended change, page styling, theme toggle, navigation, and clipboard behavior.

Expected result: both jobs succeed and the live site reflects the target commit. Check Actions logs for the resolved Node 22 patch version if Angular rejects the environment.

The workflow only builds and deploys the showcase. It does not build the distributable library, run automated tests, publish npm packages, or synchronize Figma. New pushes can cancel an in-progress Pages run because the workflow uses a shared concurrency group.

9. Release an npm package update

9.1 Prepare a versioned release

Use this workflow after component work is merged. Examples assume 0.1.3 is an unused next version; check first.

```
git switch main
git pull --ff-only origin main
git status --short
npm view ailabs-design-system versions --json
git switch -c release/0.1.3
npm version 0.1.3 --no-git-tag-version
```

1. Begin only with a clean worktree. The version command updates package metadata and lockfile without creating a commit/tag.
2. Use a patch for compatible fixes and a minor for additive changes under the team's chosen policy. Before 1.0, explicitly document breaking changes; consumers must not infer stability from a small version number.
3. Update the README, technical reference, guide, and release notes. Add a changelog if adopting one; none exists in the baseline.

4. Confirm package ownership and approved licensing metadata before public publication. The baseline has no license file.

See [npm version](#) for the version command's behavior.

9.2 Validate the release candidate

In the repository root:

```
npm ci
npm run pack
npm run build:pages
npm pack ./dist/ailabs-design-system --dry-run
```

1. Inspect the packed file list. Expect compiled JavaScript, declarations, CSS under `styles`, `schematics`, and package metadata.
2. Confirm the generated package name, version, Angular peer range, exports, and schematic path in `dist/ailabs-design-system/package.json`.
3. Install `dist/ailabs-design-system-0.1.3.tgz` in a separate Angular 22 app and verify imports, styles, and events as in Section 5.
4. Exercise `ng add` in a disposable workspace when changing the schematic; confirm the stylesheet is added once to the intended projects.
5. Commit the version and documentation changes, push `release/0.1.3`, open a PR, and merge after review. Do not commit generated dist output.

9.3 Publish from the reviewed commit

After the release PR is merged, update your clean `main` checkout and rebuild the release archive from it:

```
git switch main
git pull --ff-only origin main
git status --short
node -p "require('./package.json').version"
npm ci
npm run pack
npm run build:pages
npm login
npm whoami
npm publish ./dist/ailabs-design-system-0.1.3.tgz --dry-run
```

Confirm the printed version is exactly 0.1.3, inspect the dry-run contents, and validate this rebuilt archive in the consumer app. The dry run does not prove publishing permission. When the release artifact is verified, publish that exact archive:

```
npm publish ./dist/ailabs-design-system-0.1.3.tgz --access public
```

Complete npm's required interactive authentication. Do not publish from the repository root: the root is the development workspace, while the archive contains the built Angular package. A name/version already published cannot be reused. See [npm publish](#) and [npm publishing authentication](#).

9.4 Verify and record the release

```
npm view ailabs-design-system@0.1.3 version
npm view ailabs-design-system dist-tags --json
npm view ailabs-design-system@0.1.3 dist.integrity
git tag -a v0.1.3 -m "Release 0.1.3"
git push origin v0.1.3
```

1. Tag the exact reviewed commit used to build the published archive. Do not reuse or move an existing tag.

2. Create a GitHub Release for `v0.1.3` using the repository Releases screen, and include changes, migration notes, Figma revision, and validation results.
3. Install the registry version in a test consumer with `npm install ailabs-design-system@0.1.3` and rebuild.
4. Confirm Pages and Figma publication separately, then provide upgrade instructions to consumers.

Expected result: npm reports the new version, the Git tag identifies its source, and a fresh consumer can install and use it. The current repository has no npm publishing workflow; these are manual release steps.

10. Upgrade a consuming application

1. In the consuming app, create a branch and review the release's migration notes.
2. Inspect your installed version and the target's peers.

```
npm ls ailabs-design-system
npm view ailabs-design-system@0.1.3 peerDependencies
npm install ailabs-design-system@0.1.3
```

3. Check that `angular.json` still contains the stylesheet once. A normal npm upgrade does not rerun `ng-add` or perform component migrations.
4. Update renamed inputs, tokens, or behavior according to the release notes.
5. Run the consuming application's build and available tests. Review both themes and screens using the changed components.
6. Commit `package.json`, `package-lock.json`, and any migration edits together, then use the app's normal release process.

`npm update ailabs-design-system` follows the existing declared version range; it is not an instruction to adopt any newer release. An explicit reviewed version makes the upgrade intentional. See [npm update](#).

To pin an exact version in the manifest, use `npm install --save-exact ailabs-design-system@0.1.3`. Preserve the lockfile either way.

11. Update npm and project dependencies

11.1 Understand the three different updates

Action	What it changes
<code>npm publish</code> of this library	A registry release that consumers may adopt
<code>npm update</code> / <code>npm install</code> in a project	Its installed dependency graph and potentially manifest/lockfile
Installing another npm CLI version	The package-management tool on the developer machine

11.2 Update this repository's dependencies

1. Create a maintenance branch from current `main`.
2. Run `npm outdated` and inspect the packages requiring updates. A nonzero exit can simply indicate outdated packages.
3. For Angular upgrades, follow the [Angular update guide](#) and check the [compatibility matrix](#). Review CLI, build, compiler, core/common, platform-browser, ng-packagr, TypeScript, and Zone.js together.

4. Use `npx ng update` to inspect available framework updates. Select a compatible target deliberately instead of applying an unreviewed major upgrade.
5. Update other packages individually with `npm install <package>@<reviewed-version>` or the appropriate `--save-dev` option.
6. Inspect `package.json` and `package-lock.json`, then run `npm ci`, library and Pages builds, and the consumer archive checks.
7. If supported Angular majors change, update `peerDependencies` and the documentation only after verification. Merge through a reviewed PR.

The baseline pins Angular toolchain versions exactly. `npm update` alone will not move exact pins to a new version. Keep the lockfile; do not delete it as a routine update step. Review audit findings individually before applying fixes that may change APIs.

11.3 Update the npm CLI itself

1. Record `node --version` and `npm --version`.
2. Check the intended npm release's Node requirement with `npm view npm@<target-version> engines`.
3. Use your Node version manager or approved machine setup to install that compatible npm release. If your Node installation permits global package management, the explicit command is `npm install --global npm@<target-version>`.
4. Recheck versions and run the project's clean install and builds.

Avoid introducing a different npm CLI version as an incidental side effect of a library release. Document toolchain changes with the maintenance PR.

12. Validation checklist and release handoff

12.1 Manual acceptance checklist

Area	Check	Passing result
Installation	Fresh Angular 22 app, CSS configured	No peer, import, or stylesheet errors
Library packaging	Build and inspect archive	Public exports, CSS, and schematics included
Visuals	Light/dark, 1440/900/600/375 px widths	Readable controls, no unintended overflow
Keyboard	Tab, activation, focus visibility	Actions reachable and named
Forms	Input, checkbox, switch, radio group	Parent state matches user actions
Search	Enter, clear, back, voice, result select	Expected outputs handled by consumer
Playground	All component choices and Copy code	Preview and snippet agree
Theme	Toggle and reload	Theme changes and persists when storage works
Deployment	Pages run and live URL	Correct commit and asset base path
Figma	Library update in a sample consumer file	Intended assets and overrides verified
npm	Install the published version	Fresh consumer build succeeds

Date Calendar is a fixed March demonstration; month arrows have no navigation logic. Time Clock emits no change events and input mode is not an editable form. Search Panel substitutes demo results for an empty array. Treat these as known limitations, not acceptance criteria for full production widgets.

No test or lint scripts exist in the baseline. Builds verify compilation and packaging; they do not establish complete accessibility or behavioral correctness. Add meaningful automated tests as part of production hardening.

12.2 Record a release handoff

For every release, record: package version; Git commit and tag; PR link; Figma file and revision; affected components/tokens; compatibility and migration notes; build results; consumer installation result; npm integrity/version check; Pages run URL; known limitations; and release owner/date.

Completion sequence: design and code reviewed -> versioned release merged -> archive verified -> npm published -> source tagged -> Pages verified -> Figma published/verified -> consumers upgrade. Coordinate the Figma publication timing if design consumers need an earlier preview.

13. Troubleshooting and recovery

13.1 Common problems

Symptom	Check and resolution
npm package/version not found	Check registry and published versions; use a verified release or local archive
Angular peer dependency conflict	Align consumer Angular version with the selected package's peer range
Unknown element/property	Import the correct standalone component; check selector and input spelling
Components have no styles	Add combined stylesheet to the correct app/configuration; restart server
Copied icons are blank	Supply the referenced SVG symbols or replace the icons
ng add cannot find angular.json	Run from an Angular workspace root
ng add project missing	Use an application project key, not the repository or library name
CSS already configured message	Expected idempotent behavior; inspect ordering if visuals differ
FormControlName fails	Controls lack ControlValueAccessor; use explicit bindings or an adapter
Copy does nothing	Use HTTPS/localhost and check clipboard permissions and browser console
Theme resets	Local storage may be blocked; verify ailabs-theme in browser storage
Figma library absent	Confirm source publication, source access, destination edit rights, and plan
No Figma update arrives	Confirm the change was published and the instance remains linked
Pages shows blank screen	Check Actions logs, published artifact path, and exact base href casing
npm publish denied	Confirm account identity, package rights, registry, version, and authentication
Published version already exists	Choose a new version, rebuild, revalidate, and release again

Symptom	Check and resolution
Radio model disagrees with display	Keep one selected value in parent; handle each option's selection event

13.2 Roll back a consuming application

1. Identify the last verified compatible package version.
2. In an app maintenance branch, reinstall that version; for example, `npm install --save-exact ailabs-design-system@0.1.2` if 0.1.2 was the verified release.
3. Revert incompatible consumer code changes, rebuild, and test.
4. Commit manifest, lockfile, and code together; deploy through the app's normal process.

13.3 Recover a bad library release or Pages deployment

1. Record the faulty version/commit and impact. Keep published history intact.
2. Fix or revert the source in a new branch, review it, and merge a new commit. For a normal commit, `git revert <bad-commit>` creates a reversal; merge commits require deliberate mainline selection.
3. The merge to `main` redeploys Pages. Verify the resulting run and website.
4. For npm, publish a corrected new version; do not attempt to overwrite an existing name/version.
5. If needed, a package owner can deprecate the faulty version with `npm deprecate ailabs-design-system@<bad-version> "Use <fixed-version> instead"` after confirming the replacement is available. See [npm deprecate](#).
6. Give consumers a specific upgrade or rollback version and migration steps.

Changing an npm dist-tag does not repair apps already pinned or locked to a faulty release. Consumers must update their installed dependency.

13.4 Recover a Figma change

1. Identify the last correct source revision and affected assets.
2. Restore or correct those assets in the source file, review the result, and publish a new library update.
3. Open the affected destination files, review available updates, and apply the correction.
4. Verify instance overrides and layouts. A Git or npm rollback does not roll back Figma designs.

14. Evidence and maintenance of this guide

This guide was checked against `package.json`, `angular.json`, `ng-package.json`, `.github/workflows/deploy-pages.yml`, `schematics/ng-add`, the public API, component sources, and the showcase source at baseline commit 88d2cc8. The library and Pages builds passed during this documentation task (the Pages build required execution outside the restricted sandbox); it did not publish, push, or change Figma.

External instructions were consulted on 22 September 2026. Follow the linked official pages when platform UI labels or authentication requirements change. Keep this guide and the technical reference aligned with the package version, public APIs, and deployment workflow.